



UMAG
Universidad de Magallanes

Informe del Proyecto 1 - Reproductor de música

MUB - Digital Music Player

Integrantes:

Nicolás Balic

Tomás Minte

Daniel Uribe

Fecha: 3 de octubre de 2026

Profesora: Jacqueline Aldridge

Ayudante: Milton Hernández

Asignatura: Estructura de Datos

Índice

1. Introducción	1
2. Objetivos	2
2.1. Objetivo general	2
2.2. Objetivos específicos	2
3. Organización del proyecto	3
3.1. Estructura del repositorio	3
3.2. Compilación y ejecución	3
3.3. Convenciones de código	3
4. Desarrollo	4
4.1. Estructura principal de la canción	4
4.2. Gestión de memoria mediante arreglos dinámicos	4
4.3. Generación aleatoria y mezcla Fisher-Yates	5
4.4. Persistencia en CSV	6
4.5. Comparadores genéricos	6
4.6. Ordenamiento iterativo (Bubble Sort)	7
4.7. Ordenamiento recursivo (Quick Sort)	8
4.8. Búsqueda binaria recursiva	9
4.9. Gestión de la fila de reproducción y del historial	10
5. Pruebas y casos límite	11
6. Organización del equipo	11
7. Declaración de IA	12
8. Conclusión	13

1. Introducción

El presente informe documenta el diseño y desarrollo de **MUB Digital Music Player**, un sistema modular programado en *lenguaje C* que simula las operaciones esenciales de un reproductor de música moderno, como la gestión del catálogo, la lista de reproducción y el historial. El principal desafío consiste en estructurar todas las operaciones de almacenamiento y manipulación **exclusivamente mediante arreglos estáticos y dinámicos**. A lo largo del proyecto se construyó un catálogo musical generado automáticamente con datos aleatorios, como ID, título, artista, álbum, género, año, duración y cantidad de reproducciones.

Para asegurar un rendimiento óptimo, el sistema MUB incluye algoritmos de ordenamiento iterativos (**Bubble Sort**) y recursivos (**Quick Sort**), que permiten organizar el catálogo según múltiples criterios, junto con una **búsqueda binaria recursiva** para recuperar canciones por ID, nombre o artista. Además, el sistema mantiene una fila de reproducción manipulable por el usuario, registra las canciones escuchadas en un historial, calcula estadísticas de popularidad mediante un ranking de reproducciones y permite guardar y exportar la información en archivos CSV.

Este informe detalla las decisiones de diseño tomadas por el equipo, el funcionamiento del sistema y la implementación de los distintos algoritmos, e incluye un análisis de complejidad, mediciones de tiempo y las pruebas realizadas.

2. Objetivos

2.1. Objetivo general

Diseñar e implementar en lenguaje C un sistema de software modular que simule el funcionamiento de un reproductor de música, administre colecciones de datos, filas e historiales de reproducción exclusivamente mediante arreglos e integre algoritmos de ordenamiento y búsqueda binaria.

2.2. Objetivos específicos

- **Manipular** eficientemente arreglos dinámicos para el almacenamiento y acceso al catálogo de canciones, la lista de reproducción y el historial.
- **Desarrollar** algoritmos de ordenamiento iterativos y recursivos, además de un algoritmo de búsqueda binaria recursiva.
- **Construir** la lógica propia de un reproductor convencional, incluyendo la mezcla aleatoria de canciones, el ranking de reproducciones y la importación y exportación de datos mediante archivos CSV.
- **Aplicar** buenas prácticas de programación y modularización de código.

3. Organización del proyecto

3.1. Estructura del repositorio

El código se divide en un módulo por responsabilidad, con los encabezados en `inc/` y las implementaciones en `src/` (Tabla 1). Las carpetas `build/` y `obj/` son generadas por el `Makefile` y se excluyen del control de versiones.

Tabla 1: Módulos del sistema.

Archivo	Responsabilidad
<code>main.c</code>	Menú de línea de comandos y flujo principal del programa.
<code>creacion.c</code>	Generación aleatoria de títulos, artistas, álbumes, géneros, duraciones y reproducciones.
<code>archivos.c</code>	Lectura, escritura y exportación de CSV, validación de datos y mezcla Fisher-Yates.
<code>algoritmos.c</code>	Bubble Sort, Quick Sort, búsqueda binaria, comparadores y ranking.
<code>codes_mub.c</code>	Fila de reproducción, historial, estadísticas por género y liberación de memoria.
<code>print.c</code>	Funciones de impresión, menús y animaciones de la terminal.

3.2. Compilación y ejecución

El proyecto incluye un `Makefile` para entornos Linux. Con `make` se compila (banderas `-Wall -Wextra -Wpedantic -O3`) y deja el ejecutable en `build/mub.out`; con `make run` se ejecuta y con `make clean` se eliminan los archivos generados. Repositorio: [MUB Digital Music Player en GitHub](#).

3.3. Convenciones de código

Como parte de las buenas prácticas, se decidió que todas las funciones utilicen `PascalCase_Con_Guion_Bajo`, las variables `snake_case` y las constantes MAYUSCULAS. Cada función está documentada con su propósito, sus parámetros y su valor de retorno mediante *Doxygen Documentation Generator*. Por compatibilidad entre compiladores, no se utilizaron letras con tilde ni la letra «ñ» en el código; esta última se reemplazó por «nh», como en `anho`.

4. Desarrollo

4.1. Estructura principal de la canción

La arquitectura de **MUB** se fundamenta en el uso de arreglos y memoria dinámica. Toda la información del catálogo se centraliza en una estructura de C que representa cada registro de forma independiente y encapsula sus distintos tipos de datos, como se muestra en el Código 1.

```
1 typedef struct _cancion
2 {
3     int id;
4     char* nombre;
5     char* artista;
6     char* album;
7     char* genero;
8     int duracion_seg;
9     int anho;
10    int n_reproducciones;
11 }Cancion;
```

Código 1: Estructura principal de una canción (Ubicada en inc/codes_mub.h)

Los campos de texto son punteros a cadenas reservadas dinámicamente. Esto evita desperdiciar memoria con arreglos de tamaño fijo, pero implica que **asignar un Cancion a otro copia únicamente los punteros** (copia superficial). Este detalle es clave en las operaciones de la fila y el historial: ambos arreglos contienen copias de los registros que comparten las mismas cadenas con el catálogo, por lo que las cadenas se liberan una sola vez, al liberar el catálogo, evitando dobles **free**.

```
1 void Liberar_Memoria_Canciones(Cancion arr[], int cantidad_de_canciones)
2 {
3     for(int i = 0; i < cantidad_de_canciones; i++)
4     {
5         free(arr[i].nombre);
6         free(arr[i].artista);
7         free(arr[i].album);
8         free(arr[i].genero);
9     }
10 }
```

Código 2: Función para liberar la memoria utilizada por una canción (ubicada en inc/codes_mub.h)

4.2. Gestión de memoria mediante arreglos dinámicos

Para reservar bloques contiguos de memoria se implementó la función **malloc**, mientras que **free** se utilizó para liberarlos. Esto permite almacenar el arreglo de canciones, la fila de reproducción y el historial. Todos los bloques se liberan antes de que finalice el programa, evitando fugas de memoria y cumpliendo con buenas prácticas. Con la herramienta *Valgrind*

se verificó que no existieran fugas después de recorrer el catálogo, ordenarlo de distintas maneras, agregar canciones a la fila, reproducirlas y exportar el archivo CSV actualizado. Se obtuvo el siguiente resultado:

```
==17630==  HEAP SUMMARY:
==17630==      in use at exit: 0 bytes in 0 blocks
==17630==    total heap usage: 412 allocs, 412 frees, 326,544 bytes
      allocated
==17630==
==17630== All heap blocks were freed -- no leaks are possible
==17630==
==17630== For lists of detected and suppressed errors, rerun with: -s
==17630== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Código 3: Resumen de la comprobación de memoria con Valgrind

4.3. Generación aleatoria y mezcla Fisher-Yates

Para rellenar el catálogo inicial sin necesidad de ingresar datos manualmente, el reproductor genera canciones pseudoaleatorias la primera vez que se inicia el programa. Esta lógica está implementada en el archivo `src/creacion.c` y utiliza arreglos de punteros a caracteres que contienen listas de sustantivos y adjetivos. Mediante la función `rand()`, el programa selecciona y une palabras al azar para crear títulos, artistas y álbumes ficticios. A su vez, los atributos numéricos, como el año y la duración, se generan dentro de rangos lógicos; por ejemplo, se evita que una canción de *trap* tenga un año de lanzamiento incongruente, como 1950.

Antes de integrar una canción al catálogo, la función `Cancion_Es_Valida` comprueba que sus valores sean consistentes. El catálogo admite hasta 5000 canciones, según la constante `MAX_CANCIONES`. Las canciones que no cumplen estas condiciones se descartan.

Una vez generado este catálogo secuencial, es necesario desordenarlo para evitar que canciones generadas en el mismo ciclo queden agrupadas. Para ello, se implementó el algoritmo de mezcla **Fisher-Yates**, ubicado en `src/archivos.c`, que recorre el arreglo de canciones desde el último elemento hacia el primero, intercambiando el elemento actual con otro elegido en una posición aleatoria entre 0 e i . Su complejidad es $O(n)$.

```
1 void Mezclar_Fisher_Yates(Cancion arr[], int n)
2 {
3     int i, j;
4     if (n <= 1)
5         return;
6     for (i = n - 1; i > 0; i--)
7     {
8         j = rand() % (i + 1);
9         Cancion temp = arr[i];
10        arr[i] = arr[j];
11        arr[j] = temp;
12    }
13 }
```

Código 4: Algoritmo de mezcla Fisher-Yates (Ubicado en `src/archivos.c`)

4.4. Persistencia en CSV

Para asegurar la persistencia de los datos, el catálogo generado se guarda en un archivo CSV (valores separados por comas) usando `fprintf`. El archivo se genera una única vez: al iniciar el programa en ejecuciones posteriores, MUB detecta su existencia (`Archivo_Existe`) y lo carga en lugar de generar la música desde cero. Al terminar, el catálogo con las reproducciones actualizadas se exporta a un **archivo CSV distinto**, de modo que el original no se sobrescribe.

```
1 id,titulo,artista,album,genero,duracion_seg,anho,n_reproducciones
2 25,Aura tierno,Don Phantom,Single,Jazz,518,1953,26940
3 86,Runaway Love,Doc nBallinn,Lo mejor de 67,Funk,297,1983,46240
4 82,Static inge,Lit Rocket,Era de Delirio,Dubstep,560,1978,8970
5 66,Wild Waves,King Chacal,Modo Mambo,Jazz,301,1978,35810
6 12,Sunset Road,Lit Nova,After mortem,Rock,290,1983,59490
7 14,Ghost Nights,Lit Mencho,Mundo de 67,Pop,329,2014,3100
8 26,Crystal Dance,Lit Mencho,Mundo de 67,Jazz,84,2014,9090
```

Código 5: Ejemplo del catálogo exportado en formato CSV

Para cargar el catálogo en memoria se utiliza `Cargar_Catalogo_CSV`. La función lee el archivo línea por línea con `fgets`, separa los valores con `sscanf` mediante un formato estructurado y los asigna a una variable temporal de tipo `Cancion`. Antes de incorporarla al arreglo dinámico, `Cancion_Es_Valida` comprueba los rangos y descarta automáticamente los registros corruptos.

4.5. Comparadores genéricos

Para evitar la duplicación de código y los valores codificados de forma rígida, los algoritmos de ordenamiento y búsqueda delegan la comparación en dos funciones auxiliares. Estas se parametrizan mediante un `enum Tipo_Criterio` (ID, nombre, artista, álbum, género, duración, año y reproducciones) y, en el caso del ordenamiento, mediante un `enum Orden` (ascendente o descendente):

- `Comparar_Canciones_Sort(a, b, criterio, orden)` retorna 1 si *a* debe ir *después* de *b* (hay que intercambiar) y 0 en caso contrario.
- `Comparar_Canciones_Search(a, objetivo, criterio)` retorna 0 si hay coincidencia, -1 si *a* es menor que el objetivo y 1 si es mayor.

Ambas usan un `switch` sobre el criterio y `strcmp` para las cadenas. Gracias a esta decisión, **los mismos** Bubble Sort y Quick Sort ordenan por *cualquiera* de los ocho campos, y agregar un criterio nuevo solo requiere añadir un caso al `switch`.

4.6. Ordenamiento iterativo (Bubble Sort)

Se implementó el algoritmo Bubble Sort para tareas de ordenamiento de menor escala, como reordenar la fila de reproducción o aplicar filtros rápidos. Esta versión funciona con la ayuda de los *comparadores genéricos* y cuenta con la mejora de detenerse cuando no se realiza ningún intercambio (cuando la variable *swap* es igual a 0).

```
1 void Bubble_Sort (Cancion song[], int numero_canciones, Tipo_Criterio
   criterio, Orden orden)
2 {
3     int i, j, swap;
4     Cancion temp;
5     for (i = 0; i < numero_canciones - 1; i++)
6     {
7         swap = 0;
8         for (j = 0; j < numero_canciones - 1; j++)
9         {
10             if (Comparar_Canciones_Sort(song[j], song[j+1], criterio,
orden) == 1)
11             {
12                 temp = song[j];
13                 song[j] = song[j+1];
14                 song[j + 1] = temp;
15                 swap += 1;
16             }
17         }
18         if (swap == 0)
19             break;
20     }
21 }
```

Código 6: Algoritmo Bubble Sort (Ubicado en src/algortimos.c)

Como se observa en el Código 6, *Bubble_Sort* delega la evaluación en *Comparar_Canciones_Sort*. Esta función recibe las dos canciones, el criterio de ordenamiento (ID, título, artista, álbum, género, año, duración o reproducciones) y la dirección del orden. Retorna 0 si ambas están correctamente ordenadas o 1 si se requiere un intercambio (*swap*).

4.7. Ordenamiento recursivo (Quick Sort)

Bubble Sort resulta ineficiente para ordenar el catálogo completo, por lo que se integró **Quick Sort**, basado en la estrategia «dividir y vencer». El algoritmo elige un *pivote*, *particiona* el arreglo dejando a la izquierda los elementos que deben ir antes que el pivote y a la derecha los demás, y se llama recursivamente sobre ambas mitades.

```
1 void Quick_Sort(Cancion catalogo[], int low, int high, Tipo_Criterio
   criterio, Orden orden)
2 {
3     if (low < high)
4     {
5         Cancion pivot = catalogo[high]; // el ultimo elemento es el pivote
6         int i = (low - 1);
7
8         // menores al pivote a la izquierda, mayores a la derecha
9         for (int j = low; j <= high - 1; j++)
10        {
11            if (Comparar_Canciones_Sort(catalogo[j], pivot, criterio,
12                                         orden) == 0)
13            {
14                i++;
15                Cancion temp = catalogo[i];
16                catalogo[i] = catalogo[j];
17                catalogo[j] = temp;
18            }
19            // el pivote queda en su posicion definitiva
20            Cancion temp = catalogo[i + 1];
21            catalogo[i + 1] = catalogo[high];
22            catalogo[high] = temp;
23            int pi = i + 1;
24
25            Quick_Sort(catalogo, low, pi - 1, criterio, orden);
26            Quick_Sort(catalogo, pi + 1, high, criterio, orden);
27        }
28    }
```

Código 7: Algoritmo Quick Sort (Ubicado en src/algortimos.c)

4.8. Búsqueda binaria recursiva

Dado que el catálogo puede ordenarse previamente con Quick Sort o Bubble Sort, se aprovecha esa estructura ya ordenada aplicando **búsqueda binaria recursiva**. La implementación (Código 8) es genérica: depende del criterio ingresado por el usuario (ID, nombre o artista).

```
1
2 int Binary_Search (Cancion arr[], int low, int high, Cancion target,
3   Tipo_Criterio criterio)
4 {
5     if (low > high) return -1;
6
7     int mid = low + (high - low)/2;
8     int comparacion = Comparar_Canciones_Search(arr[mid], target, criterio
9 );
10
11     if(comparacion == 0) return mid;
12
13     if(comparacion > 0)
14         return Binary_Search(arr, low, mid - 1, target, criterio);
15     else
16         return Binary_Search(arr, mid + 1, high, target, criterio);
17 }
```

Código 8: Búsqueda Binaria (Ubicada en src/algortimos.c)

Esta búsqueda se apoya en `Comparar_Canciones_Search`, que retorna 0 si hay coincidencia exacta, -1 si el elemento central es menor que el objetivo y 1 si es mayor.

Precondición. La búsqueda binaria solo es correcta si el arreglo está ordenado *por el mismo criterio* que se busca. Por eso, antes de buscar por ID, nombre o artista, el catálogo se ordena primero por ese campo.

Coincidencias múltiples. Muchas canciones comparten artista, álbum o título, por lo que la búsqueda binaria encuentra *una* posición coincidente, aunque puede haber más. Por esta razón, se implementó la función `Buscar_Y_Mostrar_Coincidencias`, que se expande desde esa posición hacia la izquierda y hacia la derecha mientras el valor continúa coincidiendo. De esta manera, muestra todo el rango contiguo y el total encontrado. Esto funciona porque el arreglo está ordenado y todas las coincidencias se encuentran agrupadas.

4.9. Gestión de la fila de reproducción y del historial

Uno de los mayores desafíos del proyecto fue simular el comportamiento de una cola de reproducción y un historial utilizando exclusivamente arreglos estáticos y dinámicos. Para lograr esto, se implementaron mecanismos de desplazamiento de elementos dentro del arreglo.

Cuando el usuario decide reproducir una canción mediante la función `Reproducir_Cancion`, el sistema ejecuta los siguientes pasos:

1. **Extracción de la fila:** Se lee la canción en el índice 0 de la *playlist* (la primera en la fila).
2. **Actualización del historial:** Mediante la función `Agregar_Cancion_Historial`, todos los elementos del arreglo del historial se desplazan una posición hacia la derecha (`historial[i] = historial[i - 1]`). La canción recién reproducida se inserta en la posición 0, perdiendo la canción más antigua si el arreglo alcanza su tamaño máximo (10 canciones).
3. **Avance de la Fila:** Utilizando la función `Quitar_Primer_Cancion_Playlist`, todos los elementos de la lista de reproducción se desplazan una posición hacia la izquierda (`playlist[i] = playlist[i + 1]`), dejando el último espacio vacío y actualizando la próxima canción a reproducir en el índice 0.

La Figura 1 ilustra el efecto de una reproducción sobre la fila y el historial.

Figura 1: Reproducir la primera canción de la fila (A): la fila avanza y A pasa al inicio del historial. Ejemplo con 5 posiciones.

	[0]	[1]	[2]	[3]	[4]
Fila antes	A	B	C	–	–
Fila después	B	C	–	–	–
Historial antes	X	Y	Z	–	–
Historial después	A	X	Y	Z	–

5. Pruebas y casos límite

Se probaron los siguientes escenarios para verificar que el sistema maneja correctamente los errores y los casos límite:

Tabla 2: Casos límite probados.

Escenario	Comportamiento esperado
Reproducir con la fila vacía	Mensaje de error; no se modifica el historial.
Añadir una canción que ya está en la fila	Se rechaza con el aviso «La canción ya está en la fila».
Añadir con un ID inexistente	Se rechaza con el aviso de ID inválido.
Añadir con la fila llena	Se rechaza con el aviso de fila llena.
Quitar por posición fuera de rango	Mensaje indicando el rango válido (1 al total).
Quitar por ID menor o igual a 0, o ID ausente de la fila	Mensaje de error; la fila no cambia.
Vaciar una fila que ya está vacía	Aviso informativo.
Reproducir más de $K = 10$ canciones	El historial conserva solo las 10 más recientes.
Búsqueda de un valor inexistente	La búsqueda binaria retorna -1 y se informa que no hay coincidencias.
Búsqueda con varias coincidencias (mismo artista)	Se muestran todas las canciones contiguas y el total.
Registros del CSV con valores fuera de rango	Se descartan mediante <code>Cancion_Es_Valida</code> .

6. Organización del equipo

Para llevar a cabo el desarrollo de MUB Digital Music Player, las distintas partes del sistema, como la interfaz y la lógica del código, se distribuyeron de manera equitativa entre los miembros del equipo.

- **Nicolás Balic:** Desarrolló la gestión de los archivos CSV y colaboró en la implementación de distintos algoritmos para ordenar los arreglos.
- **Tomás Minte:** Participó en la generación de canciones y en las funciones para añadirlas a la fila de reproducción. Además, mejoró el archivo principal mediante la definición y prueba de casos límite.
- **Daniel Uribe:** Creó la estructura base del proyecto y organizó sus secciones. También realizó contribuciones importantes a las funciones de reproducción, historial y ordenamiento del código.

Cabe destacar que, de los 40 *commits* registrados en el repositorio, la distribución de las aportaciones presenta variaciones entre los integrantes del equipo. Esta diferencia responde a una situación logística particular durante el desarrollo: uno de los integrantes no tuvo acceso a su computador personal durante gran parte del proyecto. Para que pudiera avanzar y mantener el ritmo de trabajo, se utilizó un computador adicional proporcionado por otro integrante del equipo.

Asimismo, en momentos puntuales se optó por descargar y compartir directamente los archivos para trabajar de forma sincronizada. Con esta medida se buscó reducir la frecuencia de los conflictos de fusión (*merge conflicts*) en Git, aunque algunos ocurrieron de manera ocasional debido al trabajo simultáneo. Toda la programación y las correcciones realizadas mediante esta modalidad fueron integradas, probadas y posteriormente subidas al repositorio por un integrante específico.

7. Declaración de IA

Durante el desarrollo del proyecto se utilizaron herramientas de inteligencia artificial, entre ellas Claude de Anthropic y Gemini de Google. Su uso se orientó principalmente al aprendizaje, la resolución de dudas y la comprensión de errores de programación. Estas herramientas no se emplearon simplemente para copiar y pegar soluciones, sino como apoyo para comprender los problemas y mejorar el trabajo realizado.

A continuación, cada integrante declara si utilizó herramientas de inteligencia artificial durante el desarrollo del proyecto y, en caso afirmativo, cuáles y con qué fin.

Estudiante	Herramienta	Uso	Funciones o secciones involucradas
Nicolás Balic	Gemini y Claude	Explicación y apoyo para prevenir errores y advertencias, resolver dudas y comprender los algoritmos empleados.	Exportación y mejora de la carga de archivos CSV; planificación de la distribución de los algoritmos de ordenamiento; aprendizaje de Fisher–Yates y comprensión del funcionamiento de Quick Sort.
Tomás Minte	Gemini	Orientación durante la implementación, validación de datos, detección de errores y revisión del uso de memoria.	Generación y validación de canciones; asignación y creación de títulos, artistas y álbumes.
Daniel Uribe	Claude	Mejora de la presentación visual mediante colores y símbolos, resolución de dudas y separación de la impresión respecto de la lógica del programa.	Desarrollo directo de las funciones <code>Print_Opcion</code> y <code>Print_Titulo</code> , además de la reorganización de las funciones de impresión.

8. Conclusión

El desarrollo de **MUB Digital Music Player** permitió cumplir el objetivo de construir, en lenguaje C, un reproductor modular basado exclusivamente en arreglos estáticos y dinámicos. El sistema administra un catálogo de canciones, una fila de reproducción y un historial; además, genera datos pseudoaleatorios, valida sus registros y conserva la información mediante la carga y exportación de archivos CSV.

La implementación de comparadores genéricos permitió reutilizar *Bubble Sort* y *Quick Sort* para ordenar las canciones según distintos atributos y en ambos sentidos. A su vez, la búsqueda binaria recursiva demostró la importancia de mantener el catálogo ordenado por el mismo criterio de consulta. También se comprobó que la elección del algoritmo depende del contexto: *Bubble Sort* resulta apropiado para colecciones pequeñas, mientras que *Quick Sort* ofrece una alternativa más adecuada para organizar el catálogo completo.

Las pruebas realizadas confirmaron el comportamiento esperado ante filas vacías o llenas, identificadores inexistentes, registros CSV inválidos y búsquedas sin resultados o con coincidencias múltiples. Asimismo, la verificación con *Valgrind* registró 412 reservas y 412 liberaciones de memoria, sin bloques pendientes ni errores, lo que respalda la correcta administración de los recursos dinámicos.

En conclusión, el proyecto permitió integrar estructuras de datos, recursividad, persistencia, validación y gestión de memoria en una aplicación funcional. Más allá del resultado obtenido, su desarrollo fortaleció la capacidad del equipo para modularizar un programa, seleccionar algoritmos de acuerdo con el problema y evaluar su corrección mediante pruebas concretas.